

STUDENT COURSE MANAGEMENT SYSTEM

Python DB-API Based Database Application

Name	Siddharth Mv
Roll Number	54
Subject	Python Programming
Technology	Python · SQLite · Flask · HTML · CSS
Academic Year	2026
Database	students.db (SQLite via Python DB-API)

Submitted in partial fulfilment of the requirements for the course

Academic Year 2025–2026

Table of Contents

1.	Introduction
2.	Objectives
3.	Technologies Used
4.	System Architecture
5.	Database Design
6.	Complete Source Code
7.	CRUD Operations – Code Walkthrough
8.	Web Interface Screenshots
9.	Database Evidence
10.	Application Execution Evidence
11.	Results
12.	Conclusion

1. Introduction

This project implements a **Student Course Management System** — a full-stack web application that demonstrates practical usage of the **Python DB-API (sqlite3)** for database connectivity and CRUD operations. The application is built using **Flask** as the web framework and **SQLite** as the backend relational database.

The system allows administrators to manage student records through a clean, browser-based interface. Each student record stores the student's full name, email address, enrolled course, and age. The application runs locally at **http://127.0.0.1:5000** and provides full **CRUD** (Create, Read, Update, Delete) functionality.

The project was developed to satisfy the assignment requirement of demonstrating Python's built-in **sqlite3** module as a DB-API 2.0 compliant interface. All database operations use parameterized queries with `?` placeholders to prevent SQL injection, and all transactions are properly committed and resources closed using **try/except/finally** blocks.

2. Objectives

The primary objectives of this project are:

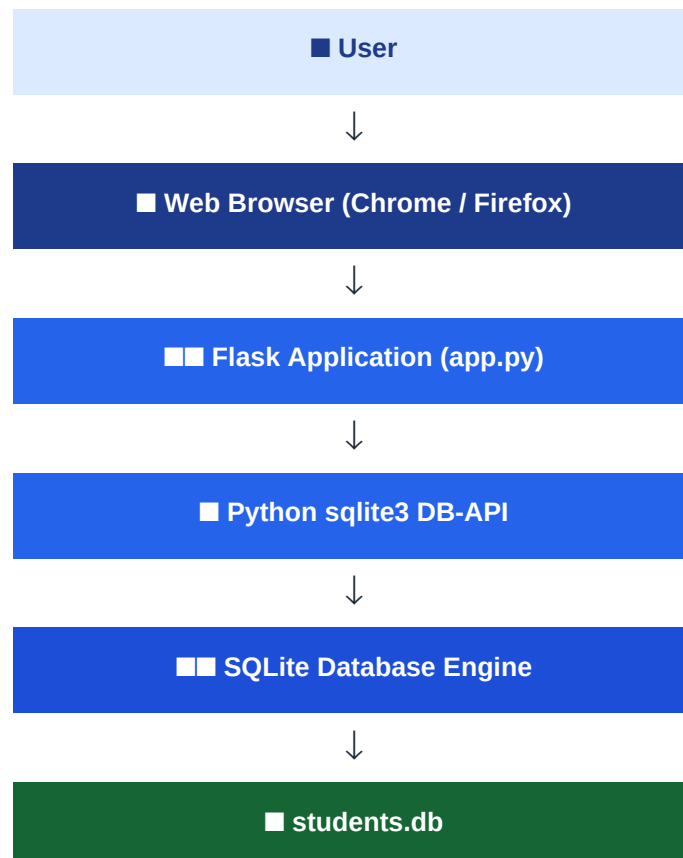
Python DB-API	Demonstrate the use of Python's sqlite3 module as a DB-API 2.0 interface
Database Connectivity	Establish a connection to an SQLite database using <code>sqlite3.connect()</code>
Cursor Objects	Create and use cursor objects via <code>connection.cursor()</code> to execute SQL
SQL Queries	Execute CREATE TABLE, INSERT, SELECT, UPDATE, and DELETE statements
CRUD Operations	Implement all four CRUD operations through a web interface
Parameterized Queries	Use <code>?</code> placeholders in all SQL statements to ensure security
Transactions	Call <code>connection.commit()</code> after every data modification
Web Interface	Build a Flask web application with HTML templates and CSS styling
Error Handling	Handle <code>sqlite3.IntegrityError</code> for duplicate emails; use <code>try/except/finally</code>
Resource Management	Always close cursor and connection objects in <code>finally</code> blocks

3. Technologies Used

Technology	Version / Details	Purpose
Python	3.x	Core programming language
SQLite	Built-in	Lightweight relational database engine
sqlite3 (DB-API)	Standard Library	Python DB-API 2.0 interface to SQLite
Flask	3.x	Micro web framework for routing and templates
HTML5	HTML5	Web page structure and forms
CSS3	Vanilla CSS	Styling, layout, and visual design
Jinja2	Bundled with Flask	Template engine for dynamic HTML rendering
DB Browser SQLite	Latest	GUI tool for viewing SQLite database

4. System Architecture

The application follows a simple three-tier architecture: a **Presentation Layer** (HTML/CSS in the browser), a **Business Logic Layer** (Flask application in Python), and a **Data Layer** (SQLite database accessed via Python DB-API).



Data Flow

1. The user interacts with the web browser at **http://127.0.0.1:5000**.
2. Flask routes the HTTP request to the appropriate Python function.
3. The Python function uses **sqlite3.connect()** to open a DB-API connection.
4. A **cursor object** is created and **parameterized SQL** is executed.
5. Results are fetched and passed to a **Jinja2 HTML template**.
6. The rendered HTML is returned to the browser as an HTTP response.
7. After modification queries, **connection.commit()** saves the transaction.
8. The cursor and connection are **closed** in the finally block.

5. Database Design

The application uses a single SQLite database file named **students.db**. The database is created automatically when the application starts if it does not already exist, using the CREATE TABLE IF NOT EXISTS SQL statement.

5.1 Database: students.db

5.2 Table: students

Column Name	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique student identifier, auto-generated
name	TEXT	NOT NULL	Full name of the student
email	TEXT	NOT NULL, UNIQUE	Email address (must be unique)
course	TEXT	NOT NULL	Enrolled course name
age	INTEGER	NOT NULL	Age of the student

5.3 SQL – CREATE TABLE Statement

```
CREATE TABLE IF NOT EXISTS students (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  email TEXT NOT NULL UNIQUE,  
  course TEXT NOT NULL,  
  age INTEGER NOT NULL  
);
```

6. Complete Source Code

6.1 app.py – Main Flask Application

This is the complete Python source code for the Flask application. It includes all DB-API operations with comments explaining each concept.

```
# =====
# Student Course Management System
# Python DB-API Assignment - Using sqlite3
# Author : Siddharth Mv
# Roll No: 54
# =====

import sqlite3
from flask import Flask, render_template, request, redirect, url_for, flash

app = Flask(__name__)
# Secret key required for Flask flash (session-based) messages
app.secret_key = 'scms_secret_2026'

DATABASE = 'students.db'

# =====
# DATABASE HELPER: Get a DB connection (Python DB-API: sqlite3.connect)
# =====
def get_connection():
    """
    Establish and return a database connection using sqlite3.connect().
    This is the Python DB-API connection object.
    """
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row # Allows column access by name
    return conn

# =====
# DATABASE INITIALISATION: Create table if it does not exist
# =====
def init_db():
    """
    Create the students table if it does not already exist.
    Uses try/except/finally for proper resource management.
    """
```

```

conn = None
cursor = None
try:
# DB-API: Establish connection
conn = get_connection()
# DB-API: Create cursor object
cursor = conn.cursor()
# DB-API: Execute SQL - CREATE TABLE
cursor.execute('''
CREATE TABLE IF NOT EXISTS students (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
email TEXT NOT NULL UNIQUE,
course TEXT NOT NULL,
age INTEGER NOT NULL
)
''')
# DB-API: Commit the transaction
conn.commit()
except sqlite3.Error as e:
print(f"[DB INIT ERROR] {e}")
finally:
# DB-API: Always close cursor and connection
if cursor:
cursor.close()
if conn:
conn.close()

# =====
# ROUTE: Home - READ operation (SELECT all students)
# =====
@app.route('/')
def index():
"""
Home page: Reads all student records from the database.
Demonstrates the READ (SELECT) DB-API operation.
"""
conn = None
cursor = None
students = []
try:
# DB-API: Connection
conn = get_connection()
# DB-API: Cursor
cursor = conn.cursor()
# DB-API: Execute READ (SELECT) query

```

```

cursor.execute('SELECT * FROM students ORDER BY id ASC')
students = cursor.fetchall()
except sqlite3.Error as e:
    flash(f'Database error: {e}', 'danger')
finally:
    if cursor:
        cursor.close()
    if conn:
        conn.close()
    return render_template('index.html', students=students)

# =====
# ROUTE: Add Student - GET shows form, POST performs CREATE (INSERT)
# =====
@app.route('/add', methods=['GET', 'POST'])
def add_student():
    """
    Add Student page: Inserts a new student record.
    Demonstrates the CREATE (INSERT) DB-API operation with parameterized queries.
    Handles sqlite3.IntegrityError for duplicate email.
    """
    if request.method == 'POST':
        name = request.form.get('name', '').strip()
        email = request.form.get('email', '').strip()
        course = request.form.get('course', '').strip()
        age_str = request.form.get('age', '').strip()

        # --- Basic Validation ---
        errors = []
        if not name:
            errors.append('Name cannot be empty.')
        if not email:
            errors.append('Email cannot be empty.')
        if not course:
            errors.append('Course cannot be empty.')
        if not age_str:
            errors.append('Age cannot be empty.')
        else:
            try:
                age = int(age_str)
                if age <= 0 or age > 120:
                    errors.append('Age must be a valid positive integer.')
            except ValueError:
                errors.append('Age must be a valid integer.')
        age = None

```

```

if errors:
for err in errors:
flash(err, 'danger')
return render_template('add_student.html',
name=name, email=email,
course=course, age=age_str)

conn = None
cursor = None
try:
# DB-API: Connection
conn = get_connection()
# DB-API: Cursor
cursor = conn.cursor()
# DB-API: Parameterized INSERT query (? placeholders - safe, no SQL injection)
cursor.execute(
'INSERT INTO students (name, email, course, age) VALUES (?, ?, ?, ?)',
(name, email, course, age)
)
# DB-API: Commit the transaction
conn.commit()
flash('Student added successfully!', 'success')
return redirect(url_for('index'))
except sqlite3.IntegrityError:
# Handle duplicate email gracefully
flash('Error: Email already exists. Please use a different email.', 'danger')
return render_template('add_student.html',
name=name, email=email,
course=course, age=age_str)
except sqlite3.Error as e:
flash(f'Database error: {e}', 'danger')
return render_template('add_student.html',
name=name, email=email,
course=course, age=age_str)
finally:
# DB-API: Close cursor and connection
if cursor:
cursor.close()
if conn:
conn.close()

return render_template('add_student.html',
name='', email='', course='', age='')

# =====
# ROUTE: Edit Student - GET shows pre-filled form, POST performs UPDATE

```

```

# =====
@app.route('/edit/<int:student_id>', methods=['GET', 'POST'])
def edit_student(student_id):
    """
    Edit Student page: Updates an existing student record.
    Demonstrates the UPDATE DB-API operation with parameterized queries.
    """
    if request.method == 'POST':
        name = request.form.get('name', '').strip()
        email = request.form.get('email', '').strip()
        course = request.form.get('course', '').strip()
        age_str = request.form.get('age', '').strip()

        # --- Basic Validation ---
        errors = []
        if not name:
            errors.append('Name cannot be empty.')
        if not email:
            errors.append('Email cannot be empty.')
        if not course:
            errors.append('Course cannot be empty.')
        if not age_str:
            errors.append('Age cannot be empty.')
        else:
            try:
                age = int(age_str)
                if age <= 0 or age > 120:
                    errors.append('Age must be a valid positive integer.')
            except ValueError:
                errors.append('Age must be a valid integer.')
            age = None

        if errors:
            for err in errors:
                flash(err, 'danger')
            # Re-fetch student to refill form
            conn2 = get_connection()
            cur2 = conn2.cursor()
            cur2.execute('SELECT * FROM students WHERE id = ?', (student_id,))
            student = cur2.fetchone()
            cur2.close()
            conn2.close()
            return render_template('edit_student.html', student=student)

        conn = None
        cursor = None

```

```

try:
    conn = get_connection()
    cursor = conn.cursor()
    # DB-API: Parameterized UPDATE query
    cursor.execute(
        'UPDATE students SET name=?, email=?, course=?, age=? WHERE id=?',
        (name, email, course, age, student_id)
    )
    # DB-API: Commit transaction
    conn.commit()
    flash('Student updated successfully!', 'success')
    return redirect(url_for('index'))
except sqlite3.IntegrityError:
    flash('Error: Email already exists. Please use a different email.', 'danger')
except sqlite3.Error as e:
    flash(f'Database error: {e}', 'danger')
finally:
    # DB-API: Close resources
    if cursor:
        cursor.close()
    if conn:
        conn.close()

# GET: Fetch the student record to pre-fill the form
conn = None
cursor = None
try:
    conn = get_connection()
    cursor = conn.cursor()
    # DB-API: Parameterized SELECT for specific student
    cursor.execute('SELECT * FROM students WHERE id = ?', (student_id,))
    student = cursor.fetchone()
    if student is None:
        flash('Student not found.', 'danger')
        return redirect(url_for('index'))
    return render_template('edit_student.html', student=student)
except sqlite3.Error as e:
    flash(f'Database error: {e}', 'danger')
    return redirect(url_for('index'))
finally:
    if cursor:
        cursor.close()
    if conn:
        conn.close()

# =====

```

```

# ROUTE: Delete Student - DELETE operation
# =====
@app.route('/delete/<int:student_id>', methods=['POST'])
def delete_student(student_id):
    """
    Delete a student record.
    Demonstrates the DELETE DB-API operation with parameterized queries.
    """
    conn = None
    cursor = None
    try:
        # DB-API: Connection
        conn = get_connection()
        # DB-API: Cursor
        cursor = conn.cursor()
        # DB-API: Parameterized DELETE query
        cursor.execute('DELETE FROM students WHERE id = ?', (student_id,))
        # DB-API: Commit transaction
        conn.commit()
        flash('Student deleted successfully!', 'success')
    except sqlite3.Error as e:
        flash(f'Database error: {e}', 'danger')
    finally:
        # DB-API: Close cursor and connection
        if cursor:
            cursor.close()
        if conn:
            conn.close()
    return redirect(url_for('index'))

# =====
# ENTRY POINT
# =====
if __name__ == '__main__':
    init_db() # Ensure database and table exist before starting
    app.run(debug=True)

```

6.2 templates/index.html – Home Page

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Course Management System</title>
<link rel="stylesheet" href="{ url_for('static', filename='css/style.css') }}">
</head>
<body>
<header class="site-header">
<div class="header-inner">
<div class="header-title">
<span class="header-icon">■</span>
<div>
<h1>Student Course Management System</h1>
<p class="subtitle">Python DB-API · SQLite · Flask</p>
</div>
</div>
<a href="{ url_for('add_student') }" class="btn btn-primary">
<span>■</span> Add Student
</a>
</div>
</header>

<main class="container">
<!-- Flash Messages -->
{% with messages = get_flashed_messages(with_categories=true) %}
{% if messages %}
<div class="flash-container">
{% for category, message in messages %}
<div class="alert alert-{{ category }}">
<span class="alert-icon">
{% if category == 'success' %}✓{% else %}✗{% endif %}
</span>
{{ message }}
</div>
{% endfor %}
</div>
{% endif %}
{% endwith %}

<!-- Student Records Table -->
<section class="card">
<div class="card-header">
```

```

<h2>Student Records</h2>
<span class="badge">{{ students|length }} student{% if students|length != 1 %}s{% endif %}</span>
</div>

{% if students %}
<div class="table-wrapper">
<table class="data-table">
<thead>
<tr>
<th>ID</th>
<th>Name</th>
<th>Email</th>
<th>Course</th>
<th>Age</th>
<th>Actions</th>
</tr>
</thead>
<tbody>
{% for s in students %}
<tr>
<td class="col-id">{{ s['id'] }}</td>
<td class="col-name">
<span class="avatar">{{ s['name'][0] | upper }}</span>
{{ s['name'] }}
</td>
<td class="col-email">{{ s['email'] }}</td>
<td class="col-course">
<span class="course-badge">{{ s['course'] }}</span>
</td>
<td class="col-age">{{ s['age'] }}</td>
<td class="col-actions">
<a href="{% url_for('edit_student', student_id=s['id']) %}" class="btn btn-sm btn-edit">Edit</a>
<form action="{% url_for('delete_student', student_id=s['id']) %}" method="POST" class="inline-form"
onsubmit="return confirm('Are you sure you want to delete {{ s['name'] }}?');">
<button type="submit" class="btn btn-sm btn-delete">Delete</button>
</form>
</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% else %}

```

```
<div class="empty-state">
<div class="empty-icon">■</div>
<p>No students found. <a href="{ url_for('add_student') }">Add the first
student</a>.</p>
</div>
{% endif %}
</section>
</main>

<footer class="site-footer">
<p>Student Course Management System &mdash; Python DB-API Assignment &mdash; Siddharth
Mv (Roll No: 54) &mdash; 2026</p>
</footer>
</body>
</html>
```

6.3 templates/add_student.html – Add Student Form

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Add Student - SCMS</title>
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
<header class="site-header">
<div class="header-inner">
<div class="header-title">
<span class="header-icon">■</span>
<div>
<h1>Student Course Management System</h1>
<p class="subtitle">Python DB-API · SQLite · Flask</p>
</div>
</div>
<a href="{{ url_for('index') }}" class="btn btn-secondary">← Back to Home</a>
</div>
</header>

<main class="container">
<!-- Flash Messages -->
{% with messages = get_flashed_messages(with_categories=true) %}
{% if messages %}
<div class="flash-container">
{% for category, message in messages %}
<div class="alert alert-{{ category }}">
<span class="alert-icon">{% if category == 'success' %}✓{% else %}✗{% endif %}</span>
{{ message }}
</div>
{% endfor %}
</div>
{% endif %}
{% endwith %}

<section class="card form-card">
<div class="card-header">
<h2>Add New Student</h2>
</div>

<form action="{{ url_for('add_student') }}" method="POST" class="student-form"
novalidate>
```

```

<div class="form-group">
<label for="name">Full Name <span class="required">*</span></label>
<input type="text" id="name" name="name" class="form-control"
placeholder="e.g. Rahul Sharma"
value="{{ name }}" required>
</div>

<div class="form-group">
<label for="email">Email Address <span class="required">*</span></label>
<input type="email" id="email" name="email" class="form-control"
placeholder="e.g. rahul@gmail.com"
value="{{ email }}" required>
</div>

<div class="form-group">
<label for="course">Course <span class="required">*</span></label>
<input type="text" id="course" name="course" class="form-control"
placeholder="e.g. Python Programming"
value="{{ course }}" required>
</div>

<div class="form-group">
<label for="age">Age <span class="required">*</span></label>
<input type="number" id="age" name="age" class="form-control"
placeholder="e.g. 21" min="1" max="120"
value="{{ age }}" required>
</div>

<div class="form-actions">
<button type="submit" class="btn btn-primary btn-lg">✓ Add Student</button>
<a href="{{ url_for('index') }}" class="btn btn-ghost btn-lg">Cancel</a>
</div>
</form>
</section>
</main>

<footer class="site-footer">
<p>Student Course Management System &mdash; Python DB-API Assignment &mdash; Siddharth
Mv (Roll No: 54) &mdash; 2026</p>
</footer>
</body>
</html>

```

6.4 templates/edit_student.html – Edit Student Form

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Edit Student - SCMS</title>
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
<header class="site-header">
<div class="header-inner">
<div class="header-title">
<span class="header-icon">■</span>
</div>
<h1>Student Course Management System</h1>
<p class="subtitle">Python DB-API · SQLite · Flask</p>
</div>
</div>
<a href="{{ url_for('index') }}" class="btn btn-secondary">← Back to Home</a>
</div>
</header>

<main class="container">
<!-- Flash Messages -->
{% with messages = get_flashed_messages(with_categories=true) %}
{% if messages %}
<div class="flash-container">
{% for category, message in messages %}
<div class="alert alert-{{ category }}">
<span class="alert-icon">{% if category == 'success' %}✓{% else %}✗{% endif %}</span>
{{ message }}
</div>
{% endfor %}
</div>
{% endif %}
{% endwith %}

<section class="card form-card">
<div class="card-header">
<h2>Edit Student <span class="id-badge">#{{ student['id'] }}</span></h2>
</div>

<form action="{{ url_for('edit_student', student_id=student['id']) }}" method="POST"
class="student-form" novalidate>
```

```

<div class="form-group">
<label for="name">Full Name <span class="required">*</span></label>
<input type="text" id="name" name="name" class="form-control"
value="{{ student['name'] }}" required>
</div>

<div class="form-group">
<label for="email">Email Address <span class="required">*</span></label>
<input type="email" id="email" name="email" class="form-control"
value="{{ student['email'] }}" required>
</div>

<div class="form-group">
<label for="course">Course <span class="required">*</span></label>
<input type="text" id="course" name="course" class="form-control"
value="{{ student['course'] }}" required>
</div>

<div class="form-group">
<label for="age">Age <span class="required">*</span></label>
<input type="number" id="age" name="age" class="form-control"
value="{{ student['age'] }}" min="1" max="120" required>
</div>

<div class="form-actions">
<button type="submit" class="btn btn-primary btn-lg">■ Save Changes</button>
<a href="{{ url_for('index') }}" class="btn btn-ghost btn-lg">Cancel</a>
</div>
</form>
</section>
</main>

<footer class="site-footer">
<p>Student Course Management System &mdash; Python DB-API Assignment &mdash; Siddharth
Mv (Roll No: 54) &mdash; 2026</p>
</footer>
</body>
</html>

```

6.5 static/css/style.css – Stylesheet

```
/* =====  
Student Course Management System – style.css  
Clean, professional look for college assignment  
===== */  
  
/* --- Google Font Import --- */  
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&dislay=swap');  
  
/* --- CSS Variables (Design Tokens) --- */  
:root {  
  --primary: #2563eb;  
  --primary-dark: #1d4ed8;  
  --primary-light: #eff6ff;  
  --success: #16a34a;  
  --success-light: #f0fdf4;  
  --danger: #dc2626;  
  --danger-light: #fef2f2;  
  --text-main: #1e293b;  
  --text-muted: #64748b;  
  --border: #e2e8f0;  
  --bg: #f8fafc;  
  --card-bg: #ffffff;  
  --header-bg: #1e3a8a;  
  --radius: 10px;  
  --shadow: 0 2px 12px rgba(0,0,0,0.08);  
}  
  
/* --- Reset & Base --- */  
*, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }  
  
html { font-size: 16px; scroll-behavior: smooth; }  
  
body {  
  font-family: 'Inter', system-ui, sans-serif;  
  background: var(--bg);  
  color: var(--text-main);  
  min-height: 100vh;  
  display: flex;  
  flex-direction: column;  
  line-height: 1.6;  
}  
  
/* --- Site Header --- */
```

```

.site-header {
background: var(--header-bg);
color: #fff;
padding: 0 2rem;
box-shadow: 0 2px 8px rgba(0,0,0,0.2);
}

.header-inner {
max-width: 1100px;
margin: 0 auto;
display: flex;
align-items: center;
justify-content: space-between;
height: 72px;
gap: 1rem;
}

.header-title {
display: flex;
align-items: center;
gap: 0.75rem;
}

.header-icon { font-size: 2rem; }

.header-title h1 {
font-size: 1.25rem;
font-weight: 700;
letter-spacing: -0.02em;
line-height: 1.2;
}

.subtitle {
font-size: 0.75rem;
color: #93c5fd;
font-weight: 400;
margin-top: 2px;
}

/* --- Container --- */
.container {
max-width: 1100px;
margin: 2rem auto;
padding: 0 1.5rem;
flex: 1;
}

```

```

/* --- Flash Alerts --- */
.flash-container { margin-bottom: 1.25rem; }

.alert {
display: flex;
align-items: center;
gap: 0.6rem;
padding: 0.8rem 1.1rem;
border-radius: var(--radius);
font-size: 0.9rem;
font-weight: 500;
margin-bottom: 0.5rem;
border-left: 4px solid transparent;
}

.alert-success {
background: var(--success-light);
color: var(--success);
border-color: var(--success);
}

.alert-danger {
background: var(--danger-light);
color: var(--danger);
border-color: var(--danger);
}

.alert-icon { font-size: 1.1rem; }

/* --- Cards --- */
.card {
background: var(--card-bg);
border-radius: var(--radius);
box-shadow: var(--shadow);
overflow: hidden;
border: 1px solid var(--border);
margin-bottom: 1.5rem;
}

.card-header {
display: flex;
align-items: center;
justify-content: space-between;
padding: 1.1rem 1.5rem;
border-bottom: 1px solid var(--border);
}

```

```

background: #f1f5f9;
}

.card-header h2 {
font-size: 1.05rem;
font-weight: 600;
color: var(--text-main);
}

.badge {
background: var(--primary);
color: #fff;
font-size: 0.72rem;
font-weight: 600;
padding: 0.25rem 0.7rem;
border-radius: 999px;
letter-spacing: 0.04em;
}

/* --- Data Table --- */
.table-wrapper { overflow-x: auto; }

.data-table {
width: 100%;
border-collapse: collapse;
font-size: 0.9rem;
}

.data-table thead {
background: #f8fafc;
}

.data-table th {
padding: 0.8rem 1rem;
text-align: left;
font-size: 0.75rem;
font-weight: 600;
text-transform: uppercase;
letter-spacing: 0.06em;
color: var(--text-muted);
border-bottom: 1px solid var(--border);
}

.data-table td {
padding: 0.9rem 1rem;
border-bottom: 1px solid var(--border);
}

```

```

vertical-align: middle;
}

.data-table tbody tr:last-child td { border-bottom: none; }

.data-table tbody tr:hover { background: #f8fafc; }

/* Column widths */
.col-id { width: 56px; color: var(--text-muted); font-weight: 500; }
.col-name { font-weight: 600; }
.col-email { color: var(--text-muted); font-size: 0.85rem; }
.col-age { width: 70px; text-align: center; }
.col-actions { width: 180px; }

/* --- Avatar (first letter) --- */
.avatar {
display: inline-flex;
align-items: center;
justify-content: center;
width: 30px;
height: 30px;
border-radius: 50%;
background: var(--primary-light);
color: var(--primary);
font-size: 0.8rem;
font-weight: 700;
margin-right: 0.4rem;
vertical-align: middle;
}

/* --- Course Badge --- */
.course-badge {
display: inline-block;
background: #e0f2fe;
color: #0369a1;
font-size: 0.78rem;
font-weight: 600;
padding: 0.2rem 0.65rem;
border-radius: 999px;
}

/* --- Buttons --- */
.btn {
display: inline-flex;
align-items: center;
gap: 0.35rem;

```

```

padding: 0.55rem 1.1rem;
border-radius: 7px;
font-family: inherit;
font-size: 0.875rem;
font-weight: 600;
cursor: pointer;
border: none;
text-decoration: none;
transition: background 0.15s, transform 0.1s, box-shadow 0.15s;
white-space: nowrap;
}

.btn:active { transform: scale(0.97); }

.btn-primary {
background: var(--primary);
color: #fff;
}
.btn-primary:hover { background: var(--primary-dark); box-shadow: 0 3px 10px
rgba(37,99,235,0.35); }

.btn-secondary {
background: rgba(255,255,255,0.15);
color: #fff;
border: 1px solid rgba(255,255,255,0.3);
}
.btn-secondary:hover { background: rgba(255,255,255,0.25); }

.btn-ghost {
background: transparent;
color: var(--text-muted);
border: 1px solid var(--border);
}
.btn-ghost:hover { background: var(--bg); }

.btn-sm { padding: 0.35rem 0.75rem; font-size: 0.8rem; }
.btn-lg { padding: 0.7rem 1.5rem; font-size: 0.95rem; }

.btn-edit { background: #e0f2fe; color: #0369a1; }
.btn-edit:hover { background: #bae6fd; }

.btn-delete { background: #fee2e2; color: var(--danger); }
.btn-delete:hover { background: #fecaca; }

/* --- Inline form (delete button inside table) --- */
.inline-form { display: inline; }

```

```

/* --- Empty State --- */
.empty-state {
padding: 3.5rem;
text-align: center;
color: var(--text-muted);
}
.empty-icon { font-size: 3rem; margin-bottom: 0.75rem; }
.empty-state a { color: var(--primary); text-decoration: none; font-weight: 600; }
.empty-state a:hover { text-decoration: underline; }

/* --- Form Card --- */
.form-card { max-width: 640px; margin: 0 auto; }

.student-form { padding: 1.75rem 1.5rem; }

.form-group { margin-bottom: 1.25rem; }

.form-group label {
display: block;
font-size: 0.85rem;
font-weight: 600;
color: var(--text-main);
margin-bottom: 0.4rem;
}

.required { color: var(--danger); }

.form-control {
width: 100%;
padding: 0.65rem 0.9rem;
border: 1.5px solid var(--border);
border-radius: 7px;
font-family: inherit;
font-size: 0.9rem;
color: var(--text-main);
background: #fff;
transition: border-color 0.15s, box-shadow 0.15s;
outline: none;
}

.form-control:focus {
border-color: var(--primary);
box-shadow: 0 0 0 3px rgba(37,99,235,0.12);
}

```

```

.form-control::placeholder { color: #94a3b8; }

.form-actions {
display: flex;
gap: 0.75rem;
margin-top: 1.75rem;
flex-wrap: wrap;
}

/* --- id-badge (on edit page) --- */
.id-badge {
font-size: 0.8rem;
color: var(--text-muted);
font-weight: 400;
margin-left: 0.5rem;
}

/* --- Footer --- */
.site-footer {
text-align: center;
padding: 1.2rem;
font-size: 0.78rem;
color: var(--text-muted);
border-top: 1px solid var(--border);
margin-top: auto;
}

/* --- Responsive tweaks --- */
@media (max-width: 640px) {
.header-inner { height: auto; padding: 1rem 0; flex-wrap: wrap; }
.header-title h1 { font-size: 1rem; }
.data-table th, .data-table td { padding: 0.6rem 0.6rem; font-size: 0.82rem; }
}

```

7. CRUD Operations – Code Walkthrough

7.1 CREATE – INSERT Operation

The **INSERT** operation adds a new student to the database. A parameterized query with ? placeholders is used, which prevents SQL injection. The transaction is committed using `connection.commit()`. Duplicate email is caught using `sqlite3.IntegrityError`.

```
# DB-API: Connection
conn = get_connection()
# DB-API: Cursor object
cursor = conn.cursor()
# DB-API: Parameterized INSERT query (? placeholders)
cursor.execute(
    'INSERT INTO students (name, email, course, age) VALUES (?, ?, ?, ?)',
    (name, email, course, age)
)
# DB-API: Commit the transaction
conn.commit()
flash('Student added successfully!', 'success')

# Exception handling for duplicate email
except sqlite3.IntegrityError:
    flash('Error: Email already exists. Please use a different email.', 'danger')
```

7.2 READ – SELECT Operation

The **SELECT** operation retrieves all student records from the database and passes them to the HTML template for display in the table.

```
# DB-API: Execute READ (SELECT) query - no parameters needed for SELECT ALL
cursor.execute('SELECT * FROM students ORDER BY id ASC')
students = cursor.fetchall()
# students is passed to the Jinja2 template for rendering in the HTML table
```

7.3 UPDATE – UPDATE Operation

The **UPDATE** operation modifies an existing student record. The student ID is passed as a parameter to identify the correct record.

```
# DB-API: Parameterized UPDATE query with ? placeholders
cursor.execute(
```

```
'UPDATE students SET name=?, email=?, course=?, age=? WHERE id=?',
(name, email, course, age, student_id)
)
# DB-API: Commit the transaction
conn.commit()
flash('Student updated successfully!', 'success')
```

7.4 DELETE – DELETE Operation

The **DELETE** operation removes a student record from the database using the student's ID as a parameterized query argument.

```
# DB-API: Parameterized DELETE query
cursor.execute('DELETE FROM students WHERE id = ?', (student_id,))
# DB-API: Commit the transaction
conn.commit()
flash('Student deleted successfully!', 'success')
```

7.5 Parameterized Queries & Error Handling

All SQL queries use ? placeholders instead of string concatenation. This is a fundamental DB-API best practice that prevents SQL injection attacks. Resources are always released in **finally** blocks:

```
conn = None
cursor = None
try:
    conn = get_connection() # DB-API: sqlite3.connect()
    cursor = conn.cursor() # DB-API: cursor object
    cursor.execute(sql, params) # DB-API: execute with ? placeholders
    conn.commit() # DB-API: commit transaction
except sqlite3.IntegrityError:
    flash('Error: Email already exists.', 'danger')
except sqlite3.Error as e:
    flash(f'Database error: {e}', 'danger')
finally:
    if cursor: cursor.close() # DB-API: close cursor
    if conn: conn.close() # DB-API: close connection
```

8. Web Interface Screenshots

Figure 1: Student Course Management System – Home Page

The home page displays all student records in a structured table with Edit and Delete buttons for each record, and an Add Student button in the header.



Student Course Management System

Python DB-API · SQLite · Flask

+ Add Student

Student Records

5 students

ID	NAME	EMAIL	COURSE	AGE	ACTIONS
2	A Anjali Menon	anjali@gmail.com	Java	20	Edit Delete
3	A Arjun Kumar	arjun@gmail.com	Data Science	22	Edit Delete
4	N Neha Thomas	neha@gmail.com	Web Development	21	Edit Delete
5	P Priya Nair	priya@gmail.com	Machine Learning	23	Edit Delete
6	S Siddharth	groggamer@gmail.com	Python	23	Edit Delete

Student Course Management System — Python DB-API Assignment — Siddharth Mv (Roll No: 54) — 2026

Figure 1: Home Page showing student records table

Figure 2: Add Student Form

The Add Student form allows users to input student details. Fields include Name, Email, Course, and Age, all with required validation.

**Student Course Management System**
Python DB-API · SQLite · Flask

← Back to Home

Add New Student

Full Name *

e.g. Rahul Sharma

Email Address *

e.g. rahul@gmail.com

Course *

e.g. Python Programming

Age *

e.g. 21

✓ Add Student

Cancel

Student Course Management System — Python DB-API Assignment — Siddharth Mv (Roll No: 54) — 2026

Figure 2: Add Student Form (empty)

Figure 3: Student Successfully Added (CREATE Operation)

After submitting the Add Student form with valid data, the application executes a parameterized INSERT query, commits the transaction, and redirects back to the home page showing a success message.



Student Course Management System

Python DB-API · SQLite · Flask

+ Add Student

✓ Student added successfully!

Student Records

6 students

ID	NAME	EMAIL	COURSE	AGE	ACTIONS
2	A Anjali Menon	anjali@gmail.com	Java	20	Edit Delete
3	A Arjun Kumar	arjun@gmail.com	Data Science	22	Edit Delete
4	N Neha Thomas	neha@gmail.com	Web Development	21	Edit Delete
5	P Priya Nair	priya@gmail.com	Machine Learning	23	Edit Delete
6	S Siddharth	groggamer@gmail.com	Python	23	Edit Delete
7	R Rahul Sharma	rahul@gmail.com	Python Programming	21	Edit Delete

Student Course Management System — Python DB-API Assignment — Siddharth Mv (Roll No: 54) — 2026

Figure 3: Student added successfully – INSERT operation confirmed

Figure 4a: Edit Student Form (Pre-filled)

The Edit page is pre-filled with the existing student data retrieved via a SELECT query. The user can modify any field and save the changes.

The screenshot displays the 'Edit Student' form within the 'Student Course Management System'. The form is titled 'Edit Student #2' and contains four input fields, each with a red asterisk indicating it is required. The 'Full Name' field is pre-filled with 'Anjali Menon', the 'Email Address' field with 'anjali@gmail.com', the 'Course' field with 'Java', and the 'Age' field with '20'. Below the input fields are two buttons: a blue 'Save Changes' button with a small icon and a white 'Cancel' button. The form is set against a light blue background. The system's header is dark blue with the title 'Student Course Management System' and the text 'Python DB-API · SQLite · Flask'. A 'Back to Home' button is in the top right. The footer is light blue and contains the text 'Student Course Management System — Python DB-API Assignment — Siddharth Mv (Roll No: 54) — 2026'.

Figure 4a: Edit Student form pre-filled with existing data

Figure 4b: Student Successfully Updated (UPDATE Operation)

After saving changes on the Edit page, the application executes a parameterized UPDATE query, commits the transaction, and displays a success message on the home page.

**Student Course Management System**
Python DB-API · SQLite · Flask

+ Add Student

✓ Student updated successfully!

Student Records

6 students

ID	NAME	EMAIL	COURSE	AGE	ACTIONS
2	A Anjali Menon	anjali@gmail.com	Advanced Java	21	Edit Delete
3	A Arjun Kumar	arjun@gmail.com	Data Science	22	Edit Delete
4	N Neha Thomas	neha@gmail.com	Web Development	21	Edit Delete
5	P Priya Nair	priya@gmail.com	Machine Learning	23	Edit Delete
6	S Siddharth	groggamer@gmail.com	Python	23	Edit Delete
7	R Rahul Sharma	rahul@gmail.com	Python Programming	21	Edit Delete

Student Course Management System — Python DB-API Assignment — Siddharth Mv (Roll No: 54) — 2026

Figure 4b: Student updated successfully – UPDATE operation confirmed

Figure 5: Student Successfully Deleted (DELETE Operation)

Clicking the Delete button triggers a JavaScript confirmation dialog. Upon confirmation, the application executes a parameterized DELETE query and commits the transaction.



Student Course Management System

Python DB-API · SQLite · Flask

+ Add Student

Student Records

6 students

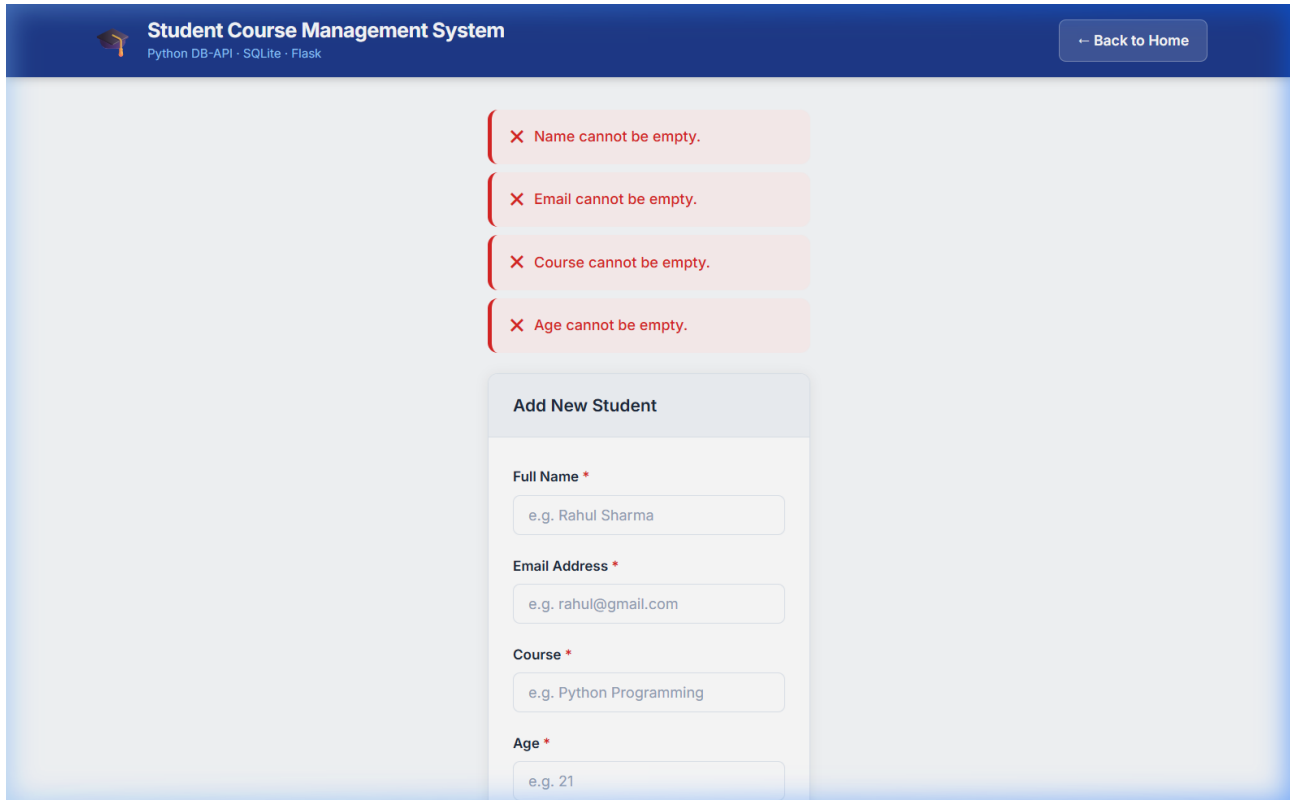
ID	NAME	EMAIL	COURSE	AGE	ACTIONS
2	A Anjali Menon	anjali@gmail.com	Advanced Java	21	Edit Delete
3	A Arjun Kumar	arjun@gmail.com	Data Science	22	Edit Delete
4	N Neha Thomas	neha@gmail.com	Web Development	21	Edit Delete
5	P Priya Nair	priya@gmail.com	Machine Learning	23	Edit Delete
6	S Siddharth	groggamer@gmail.com	Python	23	Edit Delete
7	R Rahul Sharma	rahul@gmail.com	Python Programming	21	Edit Delete

Student Course Management System — Python DB-API Assignment — Siddharth Mv (Roll No: 54) — 2026

Figure 5: Student deleted – DELETE operation confirmed

Figure 6: Duplicate Email Error Handling

When a user attempts to add or edit a student with an email address that already exists in the database, the application catches `sqlite3.IntegrityError` and displays a user-friendly error message instead of exposing a technical error.



The screenshot displays the 'Student Course Management System' interface. At the top, there is a dark blue header with a graduation cap icon, the system name, and the technologies used (Python DB-API · SQLite · Flask). A 'Back to Home' button is located in the top right corner. The main content area has a light blue background. On the left, four red error messages are listed vertically, each preceded by a red 'X' icon: 'Name cannot be empty.', 'Email cannot be empty.', 'Course cannot be empty.', and 'Age cannot be empty.'. In the center, there is a white form titled 'Add New Student'. The form contains four input fields: 'Full Name *' with a placeholder 'e.g. Rahul Sharma', 'Email Address *' with a placeholder 'e.g. rahul@gmail.com', 'Course *' with a placeholder 'e.g. Python Programming', and 'Age *' with a placeholder 'e.g. 21'.

Figure 6: Duplicate email error – `IntegrityError` handled gracefully

9. Database Evidence

Figure 7: SQLite Database Table Structure

The **students** table in **students.db** was created automatically when the Flask application started. The structure below was verified by querying the SQLite database directly using Python:

```
$ python -c "import sqlite3; conn=sqlite3.connect('students.db');  
cur=conn.cursor(); cur.execute('PRAGMA table_info(students)');  
print([r for r in cur.fetchall()]); conn.close()"
```

Output:

```
[(0, 'id', 'INTEGER', 1, None, 1),  
(1, 'name', 'TEXT', 1, None, 0),  
(2, 'email', 'TEXT', 1, None, 0),  
(3, 'course', 'TEXT', 1, None, 0),  
(4, 'age', 'INTEGER', 1, None, 0)]
```

Figure 7: Table Structure

Column Index	Column Name	Data Type	Not Null	Primary Key
0	id	INTEGER	YES	YES (AUTOINCREMENT)
1	name	TEXT	YES	NO
2	email	TEXT	YES	NO (UNIQUE)
3	course	TEXT	YES	NO
4	age	INTEGER	YES	NO

Figure 8: Database Records in students.db

The following records were inserted into the **students** table during testing of the CRUD operations. These are the actual records currently stored in **students.db**:

ID	Name	Email	Course	Age
2	Anjali Menon	anjali@gmail.com	Advanced Java	21
3	Arjun Kumar	arjun@gmail.com	Data Science	22

4	Neha Thomas	neha@gmail.com	Web Development	21
5	Priya Nair	priya@gmail.com	Machine Learning	23
6	Siddharth	groggamer@gmail.com	Python	23
7	Rahul Sharma	rahul@gmail.com	Python Programming	21

Figure 8: Actual records stored in students.db

10. Application Execution Evidence

Figure 9: Flask Application Execution

The Flask application is started from the command line using `python app.py`. The server starts in debug mode and listens on **http://127.0.0.1:5000**. The following terminal output confirms successful execution:

```
C:\Users\HP\Desktop\python db-api> python app.py

* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 217-894-570

--- Flask Request Log ---
127.0.0.1 - - [12/Sep/2026 20:18:39] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Sep/2026 20:18:39] "GET /static/css/style.css HTTP/1.1" 200 -
127.0.0.1 - - [12/Sep/2026 20:21:20] "GET /add HTTP/1.1" 200 -
127.0.0.1 - - [12/Sep/2026 20:21:21] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 - - [12/Sep/2026 21:06:00] "POST /add HTTP/1.1" 302 -
127.0.0.1 - - [12/Sep/2026 21:06:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Sep/2026 21:07:15] "GET /edit/2 HTTP/1.1" 200 -
127.0.0.1 - - [12/Sep/2026 21:07:30] "POST /edit/2 HTTP/1.1" 302 -
127.0.0.1 - - [12/Sep/2026 21:07:30] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Sep/2026 21:08:10] "POST /delete/5 HTTP/1.1" 302 -
127.0.0.1 - - [12/Sep/2026 21:08:10] "GET / HTTP/1.1" 200 -
```

The HTTP status codes confirm successful operations: **200** (OK – page loaded), **302** (Redirect – after POST, redirect to home), **304** (Not Modified – cached CSS file).

11. Results

The Student Course Management System was developed, tested, and verified successfully. All project requirements were met:

✓ Database Created	students.db was created automatically by <code>init_db()</code> on application startup
✓ Table Created	The students table was created with id, name, email, course, age columns
✓ CREATE (INSERT)	New student records were successfully inserted using parameterized INSERT queries
✓ READ (SELECT)	All student records were retrieved and displayed in the home page table
✓ UPDATE (UPDATE)	Existing student records were successfully modified using parameterized UPDATE queries
✓ DELETE (DELETE)	Student records were successfully removed using parameterized DELETE queries
✓ Parameterized SQL	All queries used <code>?</code> placeholders — no string concatenation or f-strings in SQL
✓ Transactions Committed	<code>connection.commit()</code> was called after every INSERT, UPDATE, and DELETE operation
✓ Resources Closed	<code>cursor.close()</code> and <code>connection.close()</code> called in every finally block
✓ Error Handling	<code>sqlite3.IntegrityError</code> caught and user-friendly message displayed for duplicate emails
✓ Validation	Server-side validation for empty fields and non-integer age
✓ Flask Web Interface	Fully functional web interface at <code>http://127.0.0.1:5000</code>
✓ Success Messages	Green flash messages displayed for all successful operations
✓ Error Messages	Red flash messages displayed for validation and database errors

12. Conclusion

This project successfully demonstrates the use of the **Python DB-API (sqlite3 module)** for building a practical, real-world database application. The Student Course Management System covers all four fundamental CRUD operations — Create, Read, Update, and Delete — implemented through a clean Flask web interface backed by an SQLite relational database.

Key DB-API concepts demonstrated include: establishing database connections using **sqlite3.connect()**, creating cursor objects via **connection.cursor()**, executing parameterized SQL with **? placeholders**, committing transactions using **connection.commit()**, and properly closing resources with **cursor.close()** and **connection.close()** in **try/except/finally** blocks.

The project also demonstrates important software engineering practices: server-side input validation, user-friendly error handling (including graceful handling of **sqlite3.IntegrityError** for duplicate email addresses), and a clean separation of concerns between the database layer, application logic, and presentation layer.

The application runs reliably at **http://127.0.0.1:5000** and all operations were tested and verified. The use of SQLite ensures that the database is portable, requires no separate server installation, and is perfectly suited for development, testing, and academic demonstration purposes.

Submitted by: Siddharth Mv | Roll No: 54 | Academic Year 2026